

Hello Admin,

Prepared with Pwanimate

Hello Admin,

Below is a clean, step-by-step version of the Object-Oriented Analysis & Design group assignment you shared. You can copy the entire block into a word processor (Microsoft Word, Google Docs, LibreOffice Writer, etc.) and then export it as PDF or DOCX for easy sharing with your teammates.

OBJECT-ORIENTED ANALYSIS & DESIGN

GROUP ASSIGNMENT – YEAR 2 Bed Comp-Maths, BSc Comp Sci, BSc Telecom & IT, BSc Maths, Bed Comp-Phys

1. Understanding the Existing Problem - 10 Marks

a) Identify at least five problems with the current manual student-service processes

1. Long queues – students must wait in line at multiple offices.
2. Delays in processing – manual handling slows down request fulfilment.
3. Duplication of information – the same data is entered repeatedly across offices.
4. Difficulty tracking requests – no central view of request status.
5. Limited access outside working hours – services unavailable after office hours.

(You may add more issues if you wish; the above satisfy the minimum requirement.)

b) Identify at least five stakeholders and state their roles

| Stakeholder | Role (how they interact with / are affected by OSIS) |
|-----|-----| | Student | Requests and accesses all student services (registration, fee payment, transcripts, etc.). | | Registrar / Academic Office | Approves course registrations, issues transcripts, updates academic records. | | Finance Office | Manages fee balances, processes payments, generates financial reports. | | Hostel / Accommodation Office | Handles applications for on-campus housing and room allocations. | | Examination Office | Publishes examination results and manages exam-related queries. | | (Optional) IT Support Team | Maintains the OSIS platform, ensures uptime and security. |

2. Foundations of Object Orientation - 15 Marks

Apply the four OO foundations to the proposed system.

a) Abstraction - 4 Marks

Explain how abstraction can be applied to the Student class.

- Visible to the user (public attributes/methods):
- studentID, fullName, program, currentSemester
- viewCourseSchedule(), requestTranscript(), payFees()
- Hidden from the user (private/internal details):
- dateOfBirth, nationalID, passwordHash
- calculateGPA(), updatePersonalDetails() (only staff can invoke)

Abstraction means exposing only the essential features a student needs while concealing internal data and operations that are irrelevant or sensitive to the user.

b) Encapsulation - 4 Marks

Explain how the system can protect sensitive student information.

- Store sensitive fields (e.g., passwordHash, financialDetails) as private members.
- Provide public methods that perform validation before accessing or modifying these fields (e.g., authenticateUser(), updateFinancialInfo()).

Two examples of data that should not be directly accessible by every user:

1. Password hash / authentication credentials – only the authentication module may read/write.
2. Fee balance and payment history – visible only to the student themselves and authorized finance staff.

c) Modularity - 4 Marks

Divide the proposed system into at least five modules.

| Module | Primary Responsibility | |-----|-----| | User Management | Registration, login, role-based access control. | | Course Registration | Enrolment, schedule viewing, prerequisite checking. | | Financial Services | Fee balance display, payment processing, receipts. | | Academic Records | Transcript generation, grade entry, GPA calculation. | | Accommodation Services | Hostel application, room allocation, status tracking. | | (Optional) Reporting & Analytics | Generate administrative reports, usage statistics. |

d) Hierarchy - 3 Marks

a) Identify possible inheritance relationships.

- Person (base class) → Student, Staff, Administrator
- ServiceRequest (base) → CourseRegistrationRequest, FeePaymentRequest, TranscriptRequest, AccommodationRequest

b) Explain the relationship.

- Person → Student/Staff/Administrator: All share common attributes (e.g., name, email) but each subclass adds role-specific behavior (students can request services; staff can approve; administrators can configure the system).
- ServiceRequest → specific request types: Provides a common interface for handling status, timestamps, and approval workflow, while each subclass implements details unique to that service (e.g., payment processing for FeePaymentRequest).

3. SDLC and System Analysis - 5 Marks

a) Planning phase for the OSIS project

1. Define project scope – list all services to be digitised.
2. Identify stakeholders – students, staff, finance, IT, etc.
3. Conduct feasibility study – technical, economic, operational viability.
4. Develop a high-level project charter – objectives, timeline, budget, success criteria.
5. Gather initial requirements – through interviews, surveys, and existing process documentation.

b) OSIS Analysis - answer the four “W” questions

| Question | Answer (example) | |-----|-----| | WHO? | Students, academic staff, finance officers, hostel officers, IT support. | | WHAT? | Provide online registration, fee payment, transcript requests, accommodation applications, result viewing, and general student service requests. | | WHERE? | Accessible via the university’s web portal from any internet-connected device (on-campus or remote). | | WHEN? | 24 hours a day, 7 days a week (with scheduled maintenance windows). |

c) Requirements gathering

- Functional requirements (5 examples)
 1. Student can log in using university credentials.
 2. System must allow course registration before the registration deadline.
 3. Finance module must display current fee balance and accept online payments.
 4. Students can request and download official transcripts.
 5. Staff can approve or reject accommodation applications.
- Non-functional requirements (5 examples)
 1. Security: All data transmitted over HTTPS; passwords stored as salted hashes.
 2. Usability: Interface complies with WCAG 2.1 AA for accessibility.
 3. Performance: Page load time ≤ 2 seconds under normal load.
 4. Reliability: System uptime ≥ 99.5 % per month.

5. Scalability: Able to support up to 15 000 concurrent users during peak registration periods.

How to Convert This Text into a Shareable PDF/DOC

1. Copy the entire block above (starting from “OBJECT-ORIENTED ANALYSIS & DESIGN” down to the last table).

2. Paste into your preferred editor:

- Microsoft Word → Paste → adjust heading styles if desired.
- Google Docs → Paste → the formatting will be retained automatically.
- LibreOffice Writer → Paste → you may need to apply heading levels manually.

3. Apply any additional styling you like (fonts, colors, page numbers).

4. Export / Save as PDF or DOCX:

- Word: File → Save As → Choose “PDF” or “Word Document (.docx)”.*
- Google Docs: File → Download → PDF Document (.pdf) or Microsoft Word (.docx).
- LibreOffice: File → Export As → Export as PDF (or Save As → .docx).

5. Distribute the resulting file to your group members via email, cloud storage, or the university’s learning management system.

Quick Checklist Before Sharing

- ☐ Verify that all tables display correctly.
- ☐ Ensure any personal identifiers (e.g., student IDs) have been removed or anonymised if the document will be shared publicly.
- ☐ Confirm the submission deadline (30 September 2026) is clearly visible.
- ☐ Double-check the page layout (margins, headings) after export; PDFs sometimes shift spacing.

If you need any further assistance—such as refining a particular section, adding diagrams, or preparing a presentation—just let me know!